

A Zero-Copy Apache Arrow Shared-Memory Pipeline for Low-Latency Tensor Transfer in ML Microservices

Tushar Deb Trinanku Das Sutanu Pal Jaya Dutta Souvik Nandi

Institute of Engineering and Management Kolkata, Kolkata, India

tushardebclg2002@gmail.com trinanki@gmail.com sutanu@gmail.com jaya@gmail.com souvik@gmail.com

Abstract-This study presents and measures a same-host tensor handoff design for machine-learning microservices. The prototype pairs a FastAPI producer with an Apache Arrow IPC buffer stored in shared memory and a PyTorch consumer that wraps the received numeric buffer through a NumPy view. A conventional JSON/REST implementation is evaluated under the same payload generator to provide a baseline. On an Apple M2 machine with 8.0 GiB memory, Python 3.14.3, PyArrow 23.0.1, and PyTorch 2.11.0, the Arrow-based path produced median end-to-end speedups

between 3.30x and 23.10x for payloads from 10,000 to 1,000,000 floating-point values. For payloads of at least 50,000 elements, median speedup ranged from 9.09x to 23.10x. The JSON representation required approximately 5.06x more bytes than the Arrow IPC buffer, and all integrity tests for shape, dtype, representative values, and sum error passed. These findings indicate that eliminating text serialization and avoidable memory copies can materially improve local ML service pipelines, provided the deployment can accept same-host shared-memory constraints and read-oriented tensor access.

Keywords-Apache Arrow, zero-copy, shared memory, PyTorch, FastAPI, JSON, tensor transfer, machine-learning microservices

INTRODUCTION

ML applications are often decomposed into services that generate features, embeddings, or intermediate arrays and separate workers that execute inference. That modularity is useful, but it can move the performance bottleneck from model computation to data exchange. In a JSON path, numeric arrays are formatted as text, carried through an HTTP request, parsed into Python objects, copied into typed numeric storage, and only then wrapped as tensors. As payload size grows, this conversion path consumes CPU time and memory bandwidth that do not contribute to inference.

The implementation evaluated here uses a local binary handoff. The producer casts input values into a float32 Arrow RecordBatch and writes the IPC stream into a shared-memory-backed file. The consumer opens the Arrow buffer, asks the Arrow runtime for a copy-free NumPy representation, and then creates a PyTorch tensor from that view. The design follows Arrow's emphasis on relocatable typed buffers for shared-memory access [1] and PyTorch's documented NumPy storage sharing behavior [2].

The contribution of this paper is an empirical characterization of the project implementation, not a new wire protocol. The evaluation compares the Arrow shared-memory route with a JSON/REST baseline using the same FastAPI services, payload sizes, deterministic generator, and PyTorch tensor target. The paper reports latency components, total latency, encoded payload size, and integrity outcomes so that the benefit and the limits of the design are visible.

RELATED WORK

JSON is a common API interchange choice because it is text based, broadly supported, and formally specified [3]. FastAPI also fits naturally with JSON-oriented HTTP services and OpenAPI-style interfaces [4]. Those advantages do not make JSON ideal for dense numeric vectors: each number becomes characters, parsing reconstructs native objects, and a later step must rebuild contiguous typed memory for numerical libraries.

Arrow targets a different layer of the stack. Instead of representing values as text, it specifies typed buffers, schema metadata, alignment rules, and IPC layouts for in-memory analytical data [1]. Related work has used the Arrow ecosystem for high-throughput data transfer through Arrow Flight [5] and has explored RDMA techniques for copy-minimized columnar transport [6]. The present project is smaller in scope: it stays on one host and uses shared memory rather than a network fabric.

The bridge from Arrow to PyTorch matters because the benchmark endpoint ultimately needs a tensor, not merely a serialized object. The consumer therefore converts the Arrow column to a NumPy view under zero-copy enforcement and then wraps that view with `torch.from_numpy`. PyTorch documents that the tensor and the NumPy array share storage [2], which is beneficial for read-only inference inputs but requires caution when writes or buffer lifetime are involved.

SYSTEM ARCHITECTURE

Zero-Copy Shared-Memory Tensor Transfer Architecture

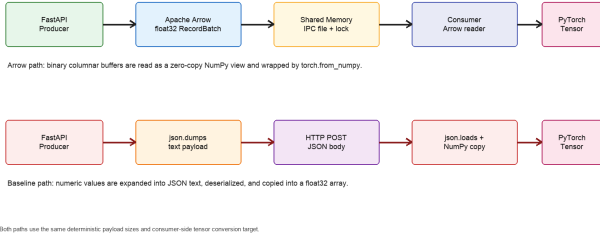


Fig. 1. System architecture comparing the Arrow shared-memory path with the JSON/REST baseline.

The two paths in Fig. 1 differ primarily in when data becomes typed contiguous memory. In the Arrow path, the producer writes a float32 RecordBatch stream to shared memory and signals readiness with a marker guarded by a file lock. The consumer reads the batch and turns the values buffer into the tensor input without materializing a separate list of Python floats. In the JSON baseline, the same values are expanded into a text body, parsed, copied into a float32 NumPy array, and then wrapped as a tensor.

METHODOLOGY

The measurements were collected on macOS-26.1-arm64-arm-64bit-Mach-O with an Apple M2 processor and 8.0 GiB memory. The software environment used Python 3.14.3, FastAPI 0.135.2, Uvicorn 0.42.0, PyArrow 23.0.1, PyTorch 2.11.0, NumPy 2.4.3, Requests 2.33.0, and FileLock 3.25.2. A temporary producer listened on localhost port 8100, a temporary consumer listened on port 8101, and the shared-memory path used /tmp/ieee_arrow_benchmark.arrow.

The benchmark used payload sizes of 10,000, 50,000, 100,000, 250,000, 500,000, and 1,000,000 floating-point elements. Each configuration was executed five times for the Arrow path and five times for the JSON path. The benchmark endpoints generated deterministic data with NumPy default_rng(42). Arrow measurements included producer conversion, shared-memory write, consumer read, consumer tensor conversion, total end-to-end latency, and buffer bytes. JSON measurements included text serialization, HTTP request time, consumer deserialization, tensor conversion, total end-to-end latency, and payload bytes.

An additional correctness run checked that the communication path preserved the tensor input expected by downstream inference code. Deterministic float values were written through the Arrow writer, read with the project consumer, and compared with an equivalent JSON-to-NumPy-to-PyTorch construction. The checks covered tensor length, torch.float32 dtype, the first five float32-cast values, and relative error in the accumulated sum.

RESULTS

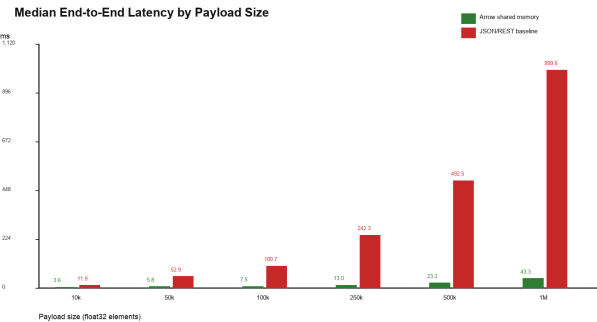


Fig. 2. Median end-to-end latency for Arrow shared memory and JSON/REST.

TABLE I

END-TO-END LATENCY AND PAYLOAD SIZE SUMMARY

Payload	Arrow median ms	JSON median ms	Median speedup	JSON/Arrow bytes
10,000	3.60	11.91	3.30x	5.03x
50,000	5.83	52.94	9.09x	5.06x
100,000	7.46	100.72	13.50x	5.06x
250,000	13.01	242.32	18.63x	5.07x

Payload	Arrow median ms	JSON median ms	Median speedup	JSON/Arrow bytes
500,000	23.17	492.50	21.25x	5.07x
1,000,000	43.27	999.60	23.10x	5.07x

The latency data show a clear scaling gap between the two communication strategies. At 1,000,000 floats, Arrow completed the end-to-end path with a median of 43.27 ms, whereas JSON/REST required 999.60 ms, yielding a 23.10x median speedup. The smallest payload includes a cold-start Arrow outlier of 819.62 ms; its median was 3.61 ms and its mean was 166.83 ms. For that reason, the results emphasize medians while still acknowledging warm-up behavior.

Payload Encoding Footprint

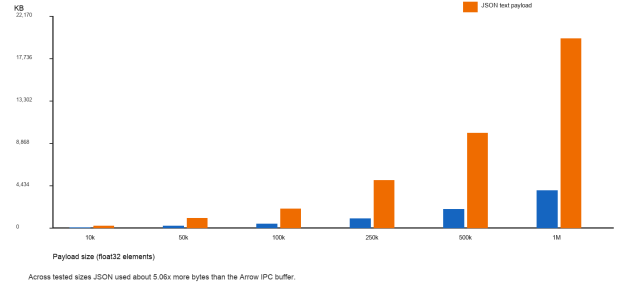


Fig. 3. Encoded payload footprint for Arrow IPC and JSON text.

TABLE II

MEDIAN COMPONENT LATENCY BREAKDOWN IN MILLISECONDS

Payload	Arrow ser.	SHM write	Arrow read	Arrow conv.	JSON ser.	JSON HTTP	JSON deser.
10k	0.20	0.78	0.46	0.0336	5.84	5.83	3.35
50k	0.85	1.06	0.52	0.0353	30.08	20.91	16.27
100k	1.60	1.15	0.46	0.0242	58.11	40.49	33.64
250k	3.83	1.39	0.61	0.0340	145.45	91.78	80.00
500k	7.79	1.42	0.74	0.0376	296.55	185.85	164.39
1M	15.88	2.30	1.05	0.0419	587.98	384.88	340.76

The component-level measurements explain the aggregate result. JSON spends most of its time formatting numbers, moving the text body through the local HTTP path, and parsing the body back into objects. Arrow still performs producer-side conversion and file-backed shared-memory writing, but its consumer-side tensor conversion is effectively constant because the tensor wraps an existing numeric view. The byte measurements are similarly consistent: the JSON representation was roughly five times larger than the Arrow IPC buffer.

TABLE III

INTEGRITY CHECK RESULTS

Payload	Shape	Dtype	First 5	Sum rel. error	Status
10,000	[10000]	torch.float32	matched	3.28e-13	passed
50,000	[50000]	torch.float32	matched	1.30e-14	passed
100,000	[100000]	torch.float32	matched	3.35e-15	passed
250,000	[250000]	torch.float32	matched	3.20e-08	passed
500,000	[500000]	torch.float32	matched	1.60e-08	passed
1,000,000	[1000000]	torch.float32	matched	8.80e-08	passed

The integrity run passed for every tested payload size. At 1,000,000 elements, the relative sum error was 8.80e-08, which is far below the 1 percent tolerance chosen for float32 accumulation effects. The first-five-value comparisons also matched after float32 casting, confirming that the benchmark measured transfer overhead rather than a lossy transformation.

DISCUSSION

The experiment supports the main engineering claim: when services are colocated and the data is already numeric, a binary shared-memory path can remove a large amount of avoidable work. JSON remains useful for control messages and interoperable APIs, but it is inefficient for repeated transport of dense tensor-like arrays. Arrow keeps the values in a typed representation, so growth in payload size mainly reflects the raw numeric buffer plus IPC metadata instead of a character-level expansion of every value.

The approach has deployment boundaries. Producer and consumer must be able to access the same shared-memory location or IPC namespace, so the design is not a drop-in replacement for ordinary cross-machine HTTP APIs. Networked settings are better served by mechanisms such as Arrow Flight or RDMA-oriented Arrow transport [5], [6]. The prototype also uses a simple file lock

and readiness file; a production service would need stronger queueing, backpressure, cleanup, and failure recovery semantics.

One implementation caveat appeared during the correctness run: PyTorch warned that the NumPy view backing the Arrow tensor was not writable. That behavior is acceptable for read-only inference inputs, which is the target use case here, but it is unsafe for consumers that mutate tensors in place. Such consumers should copy into writable storage or use a different ownership strategy.

CONCLUSION

This paper measured an Apache Arrow shared-memory IPC implementation for transferring ML tensor inputs between local microservices. For payloads between 10,000 and 1,000,000 float values, the Arrow route achieved median speedups up to 23.10x over JSON/REST and reduced encoded bytes by about 80.2 percent. Shape, dtype, representative-value, and sum-error checks all passed. The results make a practical case for using zero-copy columnar shared memory when large numeric arrays are exchanged between colocated producer and inference-worker processes.

ACKNOWLEDGMENT

The authors thank the maintainers and contributors of Apache Arrow, PyTorch, NumPy, FastAPI, Uvicorn, and related open-source tooling used to build and evaluate this prototype.

REFERENCES

- [1] Apache Software Foundation, "Arrow columnar format," Apache Arrow documentation, version 23.0.1. [Online]. Available: <https://arrow.apache.org/docs/format/Columnar.html>
- [2] PyTorch Foundation, "torch.from_numpy," PyTorch documentation, version 2.11. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.from_numpy.html
- [3] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, Internet Engineering Task Force, Dec. 2017. [Online]. Available: <https://datatracker.ietf.org/doc/rfc8259/>
- [4] S. Ramirez, "FastAPI documentation," FastAPI. [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] T. Ahmad, Z. Al Ars, and H. P. Hofstee, "Benchmarking Apache Arrow Flight: A wire-speed protocol for data transfer, querying and microservices," arXiv:2204.03032, 2022.
- [6] J. Chakraborty, M. Dorier, P. Carns, R. Ross, C. Maltzahn, and H. Litz, "Thallus: An RDMA-based columnar data transport protocol," arXiv:2412.02192, 2024.
- [7] NumPy Developers, "NumPy documentation," NumPy. [Online]. Available: <https://numpy.org/doc/stable/>
- [8] PyArrow Developers, "PyArrow documentation," Apache Arrow. [Online]. Available: <https://arrow.apache.org/docs/python/>